



An Instruction Inflation Analyzing Framework for Dynamic Binary Translators

BENYI XIE, YUE YAN, and CHENGHAO YAN, State Key Lab of Processors, Institute of Computing Technology, CAS, China and University of Chinese Academy of Sciences, China

SICHENG TAO and ZHUANGZHUANG ZHANG, University of Science and Technology of China, China

XINYU LI, YANZHI LAN, and XIANG WU, State Key Lab of Processors, Institute of Computing Technology, CAS, China and University of Chinese Academy of Sciences, China

TIANYI LIU, The University of Texas at San Antonio, USA

TINGTING ZHANG, Loongson Technology Co. Ltd., China and Institute of Computing Technology, CAS, China

FUXIN ZHANG, State Key Lab of Processors, Institute of Computing Technology, CAS, China and University of Chinese Academy of Sciences, China

Dynamic binary translators (DBTs) are widely used to migrate applications between different instruction set architectures (ISAs). Despite extensive research to improve DBT performance, noticeable overhead remains, preventing near-native performance, especially when translating from complex instruction set computer (CISC) to reduced instruction set computer (RISC). For computational workloads, the main overhead stems from translated code quality. Experimental data show that state-of-the-art DBT products have dynamic code inflation of at least 1.46. This indicates that on average, more than 1.46 host instructions are needed to emulate one guest instruction. Worse, inflation closely correlates with translated code quality. However, the detailed sources of instruction inflation remain unclear.

To understand the sources of inflation, we present *Deflater*, an instruction inflation analysis framework comprising a mathematical model, a collection of black-box unit tests called *BenchMIAOes*, and a trace-based simulator called *InflatSim*. The mathematical model calculates overall inflation based on the inflation of individual instructions and translation block optimizations. *BenchMIAOes* extract model parameters from DBTs without accessing DBT source code. *InflatSim* implements the model and uses the extracted parameters from

This is a new article, not an extension of a conference paper. The content presented in this article is intended solely for non-profit educational purposes and shall not be used for commercial applications.

This work was supported by the National Key Research and Development Program of China, project number 2022YFB3105104.

Authors' addresses: B. Xie, Y. Yan, C. Yan, X. Li, Y. Lan, X. Wu, and F. Zhang, State Key Lab of Processors, Institute of Computing Technology, CAS, 6 Kexueyuan Nanlu, Zhongguancun, Haidian, Beijing, China, 100190 and University of Chinese Academy of Sciences, No. 1 Yanchihu East Rd, Huairou, Beijing, China, 101408; e-mails: xiebenyi21b@ict.ac.cn, yanyue21s@ict.ac.cn, yanchenghao21s@ict.ac.cn, lixinyu20s@ict.ac.cn, lanyanzhi22b@ict.ac.cn, fxzhang@ict.ac.cn; S. Tao and Z. Zhang, University of Science and Technology of China, No. 96, Jinzhai Road Baohe District, Hefei, Anhui, China, 230026; e-mails: taosicheng2001@gmail.com, zhangzz2021@mail.ustc.edu.cn; T. Liu, The University of Texas at San Antonio, 1 UTSA Circle, San Antonio, Texas, USA, 78249; e-mails: tianyi.liu@utsa.edu; T. Zhang, Loongson Technology Co. Ltd., ICT Loongson Park, Daoxianghu Rd, Haidian, Beijing, China, 100095 and Institute of Computing Technology, CAS, 6 Kexueyuan Nanlu, Zhongguancun, Haidian, Beijing, China, 100190; e-mails: zhangtingting@loongson.cn.



This work is licensed under a Creative Commons Attribution-NonCommercial International 4.0 License.

© 2024 Copyright held by the owner/author(s).

ACM 1544-3566/2024/03-ART28

<https://doi.org/10.1145/3640813>

BenchMIAOes to simulate a given DBT's behavior. Deflater is a valuable tool to guide DBT analysis and improvement. Using Deflater, we simulated inflation for three state-of-the-art CISC-to-RISC DBTs: ExaGear, Rosetta2, and LATX, with inflation errors of 5.63%, 5.15%, and 3.44%, respectively for SPEC CPU 2017, gaining insights into these commercial DBTs. Deflater also efficiently models inflation for the open source DBT QEMU and suggests optimizations that can substantially reduce inflation. Implementing the suggested optimizations confirms Deflater's effective guidance, with 4.65% inflation error, and gains 5.47x performance improvement.

CCS Concepts: • **Software and its engineering** → **JIT compilers**; • **Computing methodologies** → *Modeling methodologies*;

Additional Key Words and Phrases: Dynamic binary translation, translation inflation, overhead analysis

ACM Reference Format:

Benyi Xie, Yue Yan, Chenghao Yan, Sicheng Tao, Zhuangzhuang Zhang, Xinyu Li, Yanzhi Lan, Xiang Wu, Tianyi Liu, Tingting Zhang, and Fuxin Zhang. 2024. An Instruction Inflation Analyzing Framework for Dynamic Binary Translators. *ACM Trans. Arch. Code Optim.* 21, 2, Article 28 (March 2024), 25 pages. <https://doi.org/10.1145/3640813>

1 INTRODUCTION

With the increasing popularity of virtual machines and diversity of **Instruction Set Architectures (ISAs)**, dynamic binary translation is becoming ubiquitous. Dynamic binary translation enables applications built for a guest ISA to run on a host ISA machine, with uses in several areas. First, it can translate legacy or existing ISAs to enable migration into emerging ISA ecosystems where the guest and host ISAs differ. Second, it can instrument applications like DynamoRIO [12] and Pin [37] to obtain runtime information. Third, it can profile and optimize hot paths, like Dynamo [6].

Regardless of its various purposes, translation efficiency is the primary design metric for all dynamic binary translation systems. Extensive research focuses on optimizing dynamic binary translation efficiency. Software techniques include register mapping [35, 60], indirect branch target lookup [16], arithmetic flag reduction [22, 38], enhanced translation rules [55, 61], and multi-threaded LLVM optimization [10, 64]. Hardware optimizations include **Very Long Instruction Word (VLIW)** [9, 21, 33] and ISA extensions [27, 36, 65]. These works identify specific types of translation overhead and significantly improve efficiency. Consequently, same-ISA dynamic binary translation systems like DynamoRIO and Pin demonstrate near-native efficiency, as well as similar-ISA systems like LATM [67] and MamBox64 [16, 17]. However, diverse ISA translation, especially **Complex Instruction Set Computer (CISC)** to **Reduced Instruction Set Computer (RISC)**, in ExaGear [28], Rosetta2 [1, 2], XTA [39], and LATX [67], still has noticeable overhead that prevents near-native efficiency. Our study aligns with prior work [7, 11] showing that **Dynamic Binary Translators (DBTs)** like DynamoRIO, Pin, ExaGear, Rosetta, LATX, Box64, FEX, and QEMU spend more than 98.9% of execution time on translated code for computational workloads. As Figure 1 shows, more than 99% of DynamoRIO's time is devoted to the execution of translated code. Less than 0.2% of the time involves DBT tasks like translation, disassembly, instrumentation (only for instrumentation tools), guest memory management, internal data management (e.g., branch table), and guest syscall emulation. This indicates the main overhead stems from translated code.

Since most execution time involves dynamically translated code, conventional tools can find hot code segments but struggle to determine their origin. To elucidate the origin of overhead in dynamically translated code, we utilize the term *inflation* to describe the phenomenon wherein one guest instruction is translated into multiple host instructions. Overheads in translated code can be classified into two categories: the instruction semantic gap (e.g., floating-point translation) and limitations of the DBT mechanism (e.g., indirect branch table lookup), both of which lead to one-to-multiple translation. Consequently, inflation can encompass both categories of overhead.

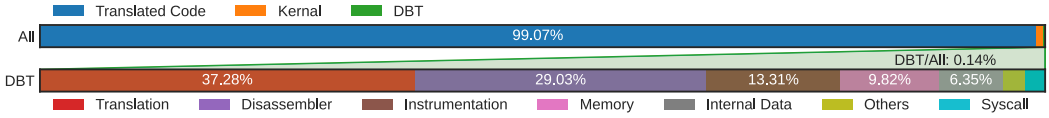


Fig. 1. DynamoRIO execution time breakdown for SPEC CPU 2017. More than 99% of DynamoRIO's time is spent executing translated code. Around 0.14% of the time involves DBT tasks.¹

We analyzed dynamic instruction inflation and performance across eight DBTs: The commercial ExaGear, Rosetta2,² LATX, and Pin, and open source DynamoRIO, Box64 [46], FEX [23], and QEMU [8]. Figure 2(a) shows the performance slowdown (DBT execution time / native execution time) per system. DynamoRIO and Pin perform best by running x86_64 guest code natively, incurring little translation overhead. Still, they have more than 1.2x slowdowns due to the limitation of the DBT mechanism. The commercial DBTs ExaGear, Rosetta2, and LATX exhibit relatively minor performance slowdowns, whereas the open source solutions Box64 and FEX display moderate slowdowns, with QEMU showcasing a substantial performance decrease. Figure 2(b) illustrates the dynamic instruction inflation for these eight DBT systems, which is correlated with the performance slowdown. Linear regression in Figure 2(c) establishes the correlation, which determines that higher inflation indicates greater performance overhead.

Despite the cross-ISA DBTs achieving relatively low inflation, the inflation remains greater than or equal to 1.46, indicating that one guest instruction is translated into at least 1.46 host instructions on average. This 46% instruction inflation highlights the persistence of significant inflation, even in commercial DBT products.

However, previous high-level studies on translation overhead provide an inadequate understanding of DBT efficiency and limited inspiration for potential optimization techniques. Thus, a comprehensive analysis methodology is needed to accurately characterize the overhead introduced by translated code. The key challenges are as follows:

- There is no off-the-shelf methodology to analyze DBT inflation at the instruction level. Instruction-level analysis is complicated by the intricate nature of DBT translation rules and optimizations.
- Analyzing overhead of commercial DBTs is limited by restricted access to their source code as commercial DBTs are typically close source. Without access to the source code, it is difficult to determine host instructions into which a specific guest instruction is translated, thus preventing further inflation analysis.
- Due to the extensive types of x86_64 instruction, it is time consuming and error prone to analyze every potential instruction. A modern disassembler [13] reveals that there are more than 1,500 types of operation code in x86_64, with possible variants for each operation code.

In this work, we address the research problem of *DBT inflation analysis methodology at the instruction level*. We also seek an open source solution to support those who cannot access the source code of commercial DBTs. To solve this problem, we present *Deflater*, an open source framework for analyzing DBT instruction inflation. This framework consists of a *mathematical model*, a

¹SPEC CPU 2017 benchmarks (ref input) are computational workloads that spend the most time executing translated code. However, the analysis of non-computational workloads, such as short-running workloads and workloads involving dynamically generated code, is beyond the scope of this work.

²Rosetta has two versions: an Ahead-Of-Time (AOT) DBT for running X86_64 macOS applications on the M-series silicon (AArch64) macOS [1], and a Just-In-Time (JIT) DBT for running X86_64 Linux applications on the AArch64 Linux virtual machine [2]. Here we use the JIT version.

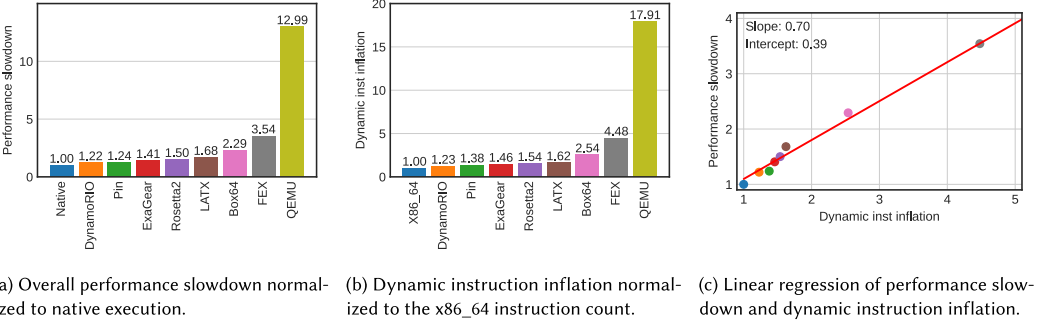


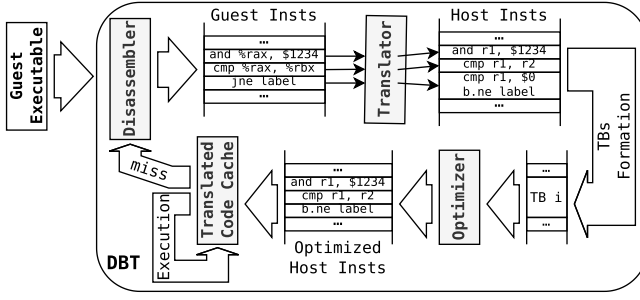
Fig. 2. The performance and dynamic instruction inflation of x86_64 SPEC CPU 2017.³

collection of black-box unit tests named *BenchMIAOes* [40], and a trace-based simulator called *InflatSim* [41]. The mathematical model calculates the overall inflation based on the inflation of individual instructions and **Translation Block (TB)** optimizations. *BenchMIAOes* extract the model parameters from DBTs without accessing their source code. *InflatSim* implements the model with extracted parameters to simulate the behavior of a given DBT. With Deflater, we simulate three commercial DBTs with inflation errors of 5.63%, 5.15%, and 3.44%, and gain insights from the simulation. In addition, using Deflater, we simulate and optimize an open source DBT with 4.65% inflation error and 5.47x performance improvement. The contributions of this work include the following:

- We propose that inflation can be represented by using a mathematical model to enable instruction-level analysis. To demonstrate this, we have developed the trace-based simulator *InflatSim* to facilitate deeper insights into DBT inflation and guide further efforts in its reduction.
- We have devised a series of meticulously designed black-box unit tests, named *BenchMIAOes* (BenchMarks for Inflation Analysis and Optimizations). These tests help to ascertain the model parameters of commercial DBTs. Additionally, to efficiently analyze the extensive types of x86_64 instructions, *BenchMIAOes* are tailored along two orthogonal dimensions: basic x86_64 instructions and variant instructions.
- We simulated the inflation of three commercial DBTs and gained insights from the simulation results. Our insights encompass x86_64-DBT-friendly ISA features, efficient translation rules, and TB optimizations.
- Furthermore, we applied Deflater to a practical development process, to optimize an open source DBT, *QEMU*. Deflater efficiently simulated the *QEMU*'s dynamic instruction inflation, effectively guided optimizations, and achieved substantial performance improvements.

The rest of this article is organized as follows. Section 2 provides an outline of the DBT. Section 3 introduces Deflater, including the mathematical model, *BenchMIAOes*, and *InflatSim*. Section 4 presents the evaluation of Deflater simulation results. Section 5 demonstrates the utilization of Deflater as a guide for *QEMU* optimizations. Section 6 provides an overview of related work on the analysis of DBT overhead. Section 7 discusses the limitations of Deflater and presents directions for future research. Section 8 summarizes this work.

³The benchmarks were statically compiled using GCC with -O3 optimization. Additionally, for X86_64, we specified the architecture as -march=x64-64 with SSE and SSE2 enabled and AVX disabled; for the ARM, we used -march=armv8-a with SIMD enabled and NEON/SVE disabled; and for LoongArch, we used -march=loongarch64 and enabled 128-bit SIMD.



(a) The architecture of a DBT.

```

1  Insn ** trans(Insn *ginsn) {
2      switch (ginsn->op_type) {
3          case X86_ADD: case X86_SUB:
4              return trans_addsub(ginsn);
5          case X86_MOV:
6              return trans_mov(ginsn);
7          case X86_CALL:
8              return trans_call(ginsn);
9          // other cases ...
10     }
11 }

```

(b) The code of a DBT's translator.

Fig. 3. The overview of a DBT. There are four main components in a DBT: disassembler, translator, optimizer, and translated code cache. The DBT processing flow can be abstracted as a loop among these four components.

2 BACKGROUND

In this section, we provide a brief introduction to the functioning of a DBT. A DBT is a type of software that enables the execution of applications designed for one ISA on another ISA platform. The term *guest* refers to the ISA platform emulated by DBT, and the term *host* refers to the ISA platform on which the DBT runs. There are two types of DBTs: user-level DBTs and system-level DBTs. User-level DBTs target applications as guests, whereas system-level DBTs target **Operating Systems (OSes)** as guests. Since our analysis focuses on user-level DBTs, all subsequent mentions of DBTs in this article specifically pertain to user-level DBTs.

Figure 3(a) illustrates the four main components of a DBT: a disassembler, a translator, an optimizer, and a translated code cache. The DBT operates through a loop involving these four components:

- (1) The *disassembler* disassembles the guest executable and creates data structures representing guest instructions.
- (2) The *translator* converts each disassembled guest instruction into corresponding host instructions. As shown in Figure 3(b), the translator code logic uses a switch-case statement to determine the translation based on its type. Similar guest instruction types, like add and sub, may utilize the same translation function. Each guest instruction is translated into one or more host instructions, The host instructions are organized into basic blocks called TBs [8, 16, 53], which have single entries and exits to facilitate optimization analysis.
- (3) The *optimizer* improves the performance of host instructions within a TB and across multiple TBs.
- (4) The optimized host instructions are executed on the host OS, and they are stored in the *translated code cache* for efficient re-execution. After executing a TB, the DBT looks up the next TB in the translated code cache. If found, execution continues seamlessly; otherwise, the preceding processing flow is repeated.

DBT implementations may vary slightly depending on optimization strategies and purposes. For example, the translated code cache can be persistent [63], translation and optimization can offload to separate threads [25], and the scope of optimization can expand from TB to trace or region [14, 57]. Despite these variations, the disassembly-translation-optimization-execution loop depicted in Figure 3(a) represents the overall processing flow for DBT implementations.

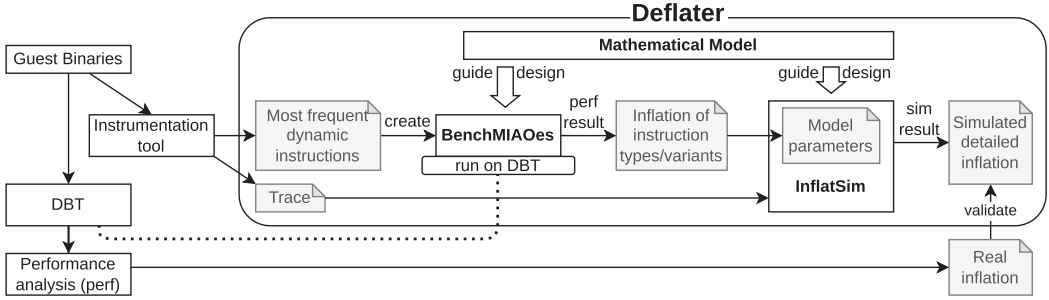


Fig. 4. Overview of the Deflater framework. The theoretical basis of Deflater is a mathematical model. The Deflater consists of a collection of black-box unit tests called *BenchMIAOes* and an inflation simulator called *InflatSim*.

3 DESIGN OF THE DEFLATER FRAMEWORK

The Deflater framework provides a comprehensive analysis of instruction inflation. It relies on a mathematical model that underpins Deflater’s two key components: the *BenchMIAOes*, a collection of black-box unit tests, and the *InflatSim*, an inflation simulator. Figure 4 outlines Deflater’s workflow:

- (1) To address the challenges of designing a multitude of *BenchMIAOes*, we selectively focus on prevalent dynamic instructions, ensuring minimal inflation errors. We only create *BenchMIAOes* to these prevalent instructions. The frequency of dynamic instructions can be obtained by an instrumentation tool.
- (2) By executing *BenchMIAOes* on the provided DBT using a performance analysis tool, we extract parameters on the inflation of instruction types and variants.
- (3) *InflatSim* leverages the extracted parameters to model inflation of the provided DBT.
- (4) We acquire guest binary traces using an instrumentation tool. *InflatSim* uses the traces to simulate inflation guided by the constructed model.
- (5) We can determine the real inflation using a performance analysis tool such as *perf*. Comparing the real inflation with simulated inflation validates our simulation. However, *perf* cannot capture detailed inflation, whereas *InflatSim* offers a comprehensive view of detailed inflation.

The remainder of this section details the mathematical mode, *BenchMIAOes*, and *InflatSim*.

3.1 Inflation Mathematical Model

Practical DBTs (DBTs for practical production and daily life, like the aforementioned ExaGear [28], Rosetta2 [1, 2], LATX [67], Box64 [46], FEX [23], and QEMU [8, 47]) usually have strict requirements for precise exceptions and adopt conservative optimizations in their real DBT products to avoid instruction boundary violations. More specifically, the memory access and branch instructions can potentially trigger memory protection violation exceptions like segmentation faults. Apart from memory exceptions, there are also arithmetic exceptions, user interrupts, and so forth, although rare in SPEC CPU 2017. Figure 5 shows that about half of the dynamic instructions in SPEC CPU 2017 are memory access and branch instructions. This large proportion limits aggressive software optimizations such as instruction rescheduling in practical DBTs, since it would be challenging to guarantee the guest’s precise exception after aggressive optimizations.

Although research exists on using hardware mechanisms to ensure the guest’s precise exceptions [7, 33], practical DBTs, especially commercial ones, target general personal computers

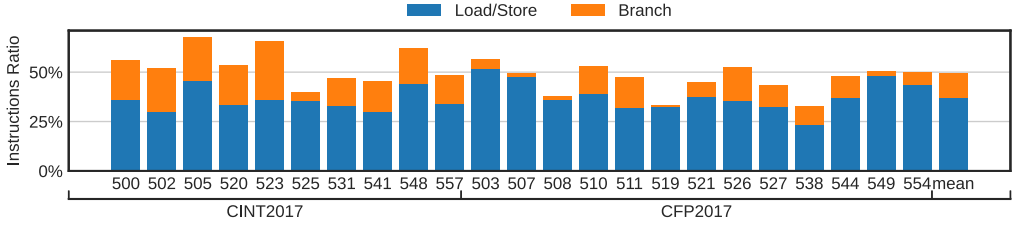


Fig. 5. Potential exception instructions in SPEC CPU 2017. Loads and stores can potentially trigger read and write exceptions. Branches can potentially trigger execution exceptions.

lacking special hardware support. Unlike practical DBTs, some DBT research focuses on aggressive optimizations and ignores the instruction boundaries. This is beyond the scope of our study in this work. Consequently, we get the following observation.

OBSERVATION 1. *To preserve the guest’s precise exception, practical software-based DBTs tend not to employ optimizations that could potentially break instruction boundaries.*

Based on Observation 1, the overall instruction inflation of an application can be represented as the weighted sum of per-instruction inflation, minus optimizations, as shown in Equation (1):

$$Inflation = \frac{\#insts_{translated}}{\#insts_{guest}} = \frac{\sum_i [\mathcal{E}inst_i \times \text{inf}(inst_i)] - \sum_j [\mathcal{E}TB_j \times \text{opt}(TB_j)] - \epsilon}{\sum_i \mathcal{E}inst_i}, \quad (1)$$

where the prefix symbol # denotes *count*, and the prefix symbol $\mathcal{E}$ denotes *the execution count of an instruction or a TB*. The function $\text{inf}()$ calculates the inflation of a single instruction, and the function $\text{opt}()$ calculates the number of optimized instructions within a TB. The symbol ϵ denotes the number of optimized instructions across TBs.

To balance model complexity and accuracy, we simplify modeling of cross-TB optimizations. For instance, a common optimization is reducing redundant arithmetic flags, and we simplify the model by assuming that all flags can be reduced. The x86_64 ISA has six flag bits in the EFLAGS register, implicitly set by arithmetic instructions. This optimization aims to prune the redundant flags calculation. To support this assumption, based on an in-house DBT, we found that dynamic instructions evaluating used flags are only 1.55% of total dynamic instructions after applying this optimization. Therefore, assuming that all flags are reduced introduces only 1.55% error. Since there are no other major cross-TB optimizations, it is reasonable to simplify cross-TB optimization modeling.

3.2 BenchMIAOes: The Black-Box Unit Tests

BenchMIAOes are specifically designed to extract the model parameters from individual instructions, and their feasibility relies on Observation 1. Figure 6 illustrates the core code of a BenchMIAO. The instructions that are to be tested are referred to as *guest_snippet*. To minimize the impact of BenchMIAO initialization overhead $\#insts_{overhead}$, the *guest_snippet* is repeated R times, as depicted in lines 2 through 4. To mitigate the impact of the DBT overhead $\#insts_{DBT}$, the repeated *guest_snippet* is executed in the LOOP of *loop_number* times.

With the help of BenchMIAO’s core code in Figure 6, the inflation of the guest snippet can be calculated. The method is shown in Equation (2):

$$Inflation_{snippet} = \frac{\#insts_{translated}}{\#insts_{snippet}} = \frac{\frac{\#insts_{dyn} - \#insts_{DBT} - \#insts_{overhead}}{loop_number} - 2}{R \times \#insts_{snippet}} \approx \frac{\frac{\#insts_{dyn}}{loop_number} - 2}{R \times \#insts_{snippet}}, \quad (2)$$

```

1  LOOP:
2  <guest_snippet>
3  /* ...repeat <R> times... */
4  <guest_snippet>
5  dec <loop_number>
6  jne LOOP

```

Fig. 6. BenchMIAO core code. The guest snippet is repeated $\langle R \rangle$ times and executed $\langle \text{loop_number} \rangle$ times in the LOOP.

Table 1. Dynamic Instructions Occupying More Than 1% (SPEC CPU 2017, Ref Input Set)

Instruction Type	Instructions	Proportion (%)
move	mov	17.32
float-point move	movss/d	14.26
conditional jump	16 kinds of jcc	9.72
float-point add and sub	addss/d, subss/d	9.50
add and sub	add, sub	8.83
compare and test	cmp, test	8.66
float-point multiply	mulss/d, mulps/d	8.40
extension	movsx, movsxd, movzx	3.95
stack operation	push, pop	3.00
address calculation	lea	2.96
logic	and, or, xor	2.13
function call	call, ret	1.67
shift and rotate	shl/r, sal/r, rcl/r, rol/r	1.46
nop in DBT	nop, endbr32/64, . . .	1.12

where $Inflation_{snippet}$ stands for the overall code inflation of *guest_snippet* program; $\#insts_{translated}$ is the dynamic instruction counts in translated code cache; $\#insts_{snippet}$ is the snippet instruction counts before translation; and $\#insts_{dyn}$ denotes the overall dynamic instruction counts, and it can be obtained through performance analysis tools, like Linux's `perf` [18], or calculated by instrumentation tools. The number 2 represents loop control instructions in lines 5 and 6 of Figure 6. The meaning of other parameters is the same as the symbols in Figure 6.

Our analysis of SPEC CPU 2017 execution reveals that despite a large number of instructions (>1,500) in x86_64, those executing more than 1% constitute a much smaller number, as illustrated in Table 1. To alleviate the burden of code writing, we decompose BenchMIAOs into two orthogonal dimensions. One dimension focuses on extracting inflation among *basic instruction types*, specifically those using full-width registers as operands, or in other words, RISC-style instructions. The other dimension concentrates on extracting extra inflation among *instruction variants*, such as memory accessing, addressing, immediate loading, and sub-registers. Additionally, we determine the frequency of occurrence for each x86_64 instruction type in guest applications, such as the SPEC CPU 2017 benchmark suite. Infrequent basic instruction types are pruned to reduce the number of BenchMIAOs. To further reduce the number of BenchMIAOs, we group the x86_64 instructions by functionality. Instructions in the same group share the same or similar inflation calculation methods. Therefore, testing only a few representative instructions in each group suffices. Table 1 shows instruction groups with proportions greater than 1% in the SPEC CPU 2017 benchmark suite using the ref input.

```

1  int main() {
2      uint64_t loop_number=20000000, one=1, out;
3      asm volatile(
4          "xor %0, %0\n\t" "LOOP:\n\t"
5          REPEAT_FIFTY("add %2, %0\n\t")
6          "dec %1\n\t" "jne LOOP\n\t"
7          : "=r"(out) : "r"(loop_number), "r"(one) :
8      );
9      printf("out %ld\n", out);
10     return 0;
11 }

```

Fig. 7. BenchMIAO example. The add instruction is repeated 50 times and executed 20 million times in a loop.

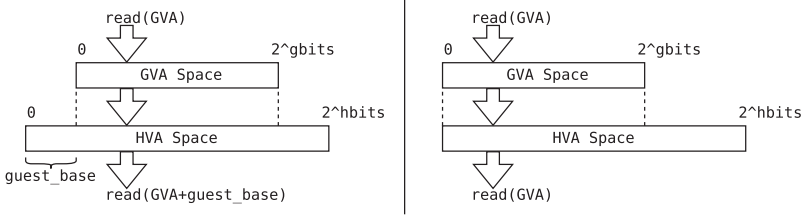


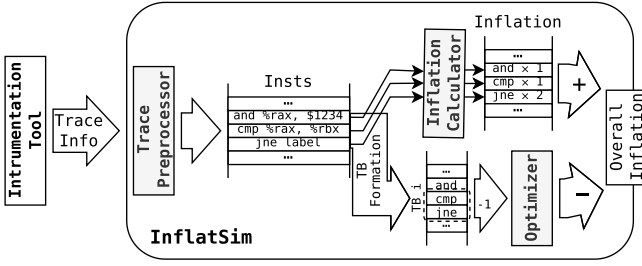
Fig. 8. Memory space mapping in DBT. Left: With a `guest_base`: one guest read is translated into one host add and one host read. Right: Without a `guest_base`: one guest read is translated into one host read.

3.2.1 Basic Instruction Types. In the BenchMIAO design, basic instruction types refer to instructions utilizing full-width registers. For example, `add %rax, %rbx` uses 64-bit registers, classified as a basic add instruction. In contrast, `add 1, %rbx` uses an immediate number, thus not deemed a basic instruction, but an immediate variant of the add instruction. Figure 7 exemplifies a BenchMIAO for basic add instruction, where the repeat times R is 50 via a C macro `REPEAT_FIFTY`, and the `loop_number` is 20 million.

3.2.2 Instruction Variants. In the BenchMIAO design, x86_64 instruction variants are divided into four categories: memory accessing, addressing, sub-register, and immediate loading.

Memory Accessing. Unlike RISC's load-calculate-store paradigm, x86_64 lacks dedicated memory access instructions. Instead, most x86_64 instructions are capable of accessing memory directly. Consequently, a memory-accessing variant is translated to a basic host instruction plus additional load/store emulation instructions. In DBT, **Guest Virtual memory Addresses (GVAs)** are linearly mapped to **Host Virtual memory Addresses (HVAs)**, as Figure 8 depicts. The linear mapping introduces a bias known as `guest_base`. Guest memory access is translated into two host instructions: an `add guest_base` instruction and a load/store instruction. High-performance DBTs typically set `guest_base=0`, enabling one-to-one mapping between the guest's and host's memory access. Our study does not focus on situations where the number of gbits exceeds the number of hbits, which is not common for practical DBTs. Memory accessing BenchMIAOs are designed to verify whether the one-to-one translation is achieved in target DBTs.

Addressing. X86_64 operands utilize an intricate algorithm to calculate memory addresses, which consists of five parts shown by Equation (3). These five parts can combine into multiple modes, called *addressing modes*. Additionally, x86_64 accommodates **Program Counter (PC)**-related addressing modes like `PC + displacement`. X86_64 employs a flat memory address space model, where segmentation is generally disabled, except for some situations like **Thread Local**



(a) The architecture of InflatSim.

```

1  int inf_calc(Insn *insn) {
2  switch (insn->op_type) {
3  case X86_ADD: case X86_SUB:
4      int inf = 1;
5      for (opr in insn->oprs) {
6          if (opr->type == MEM_READ)
7              inf += 1;
8          if (opr->type == MEM_WRITE)
9              inf -= 1;
10         // other inf calc ...
11     }
12     return inf;
13     // other cases ...
14 }
15 }

```

(b) The code of inflation calculator.

Fig. 9. The overview of InflatSim. The overall inflation is calculated by aggregating per-instruction inflation minus optimized inflation.

Storage (TLS). In contrast, RISC architectures possess much simpler memory addressing, typically just base and displacement for load and store instructions. Index and scale addressing modes necessitate extra arithmetic instructions. We design combinations of scale, index, base, displacement, and PC to expose DBT translation of addressing modes.

$$address = segment(SegReg) + base(GPR) + index(GPR) \times scale(2bit\ imm) + displacement(32bit\ imm) \quad (3)$$

Sub-Register. Due to backward compatibility, x86_64 permits accessing a portion of a **General-Purpose Register (GPR)**, referred to as a sub-register variant. Specifically, `rax`, `rbx`, `rcx`, and `rdx` can be accessed by low 8 bits [7:0], high 8 bits [15:8], 16 bits [15:0], and 32 bits [31:0]. Other GPRs support three sub-register types, except for the high type. In x86_64, writing an 8-bit or 16-bit sub-register retains higher bits unmodified, whereas writing a 32-bit sub-register sets higher bits to zero. In contrast, ISAs like AArch64 and LoongArch only support 32-bit sub-register accessing, with higher bits destroyed by sign/zero extension after writing. To detect DBTs handling of sub-register access and correctness of high bits, we design BenchMIAOes with different sub-registers as source and destination operands.

Immediate Loading. X86_64 is a variable-length ISA, with a maximum 15-byte instruction length, accommodating encoding up to 64-bit immediate directly in one instruction. In contrast, AArch64 and LoongArch are 32-bit fixed-length ISAs, incapable of directly encoding an immediate over 32 bits. If the immediate exceeds the direct encoding length, (e.g., most AArch64's load/store instructions support 9-bit immediate, and most of AArch64's arithmetic instructions support 12-bit immediate), then the immediate needs to be patched up by multiple instructions. We design various immediate loading BenchMIAOes to expose how DBTs patch up immediates.

3.3 InflatSim: The Trace-Based Simulator

InflatSim is an ISA- and DBT-agnostic trace-driven simulator. InflatSim implements the model and utilizes the extracted parameters from BenchMIAOes to simulate the behavior of a given DBT. Its processing flow is similar to that of a DBT, except that InflatSim does not emulate the functionality of each instruction and consumes trace information generated from an instrumentation tool. The instrumentation tool can be hardware based, such as Intel's Processor Trace [29] and ARM's CoreSight Trace [3], or software based, such as Pin [37], DynamoRIO [12], and QEMU TCG plugin [48]. The trace information, including PC and instruction binary code, is processed by InflatSim's three components: trace preprocessor, inflation calculator, and optimizer, as depicted in Figure 9(a):

Table 2. Information of Three Commercial CISC-to-RISC DBTs

DBT	Company	Version	Guest Applications' Platform	Host Platform
ExaGear [28]	Huawei	2.0.0.1	x86_64 Linux	Kunpeng (ARMv8.2-A) Linux
Rosetta2 [2]	Apple	289.7	x86_64 Linux	M-series (ARMv8.5-A) Linux
LATX [67]	Loongson	1.3.0	x86_64 Linux	LoongArch Linux

- (1) The *trace preprocessor* serves as the frontend for InflatSim. Unlike a DBT's disassembler decoding from an executable file, InflatSim's trace preprocessor disassembles guest instructions from trace information and prepares two essential data structures, instructions, and TBs, for the inflation calculator and optimizer.
- (2) The *inflation calculator* evaluates inflation based on the instruction type. As depicted in Figure 9(b), the outline of the inflation calculator is a switch-case statement. For example, in the case of `add %rax, (%rbx)`, which adds a register operand `%rax` to a memory operand `(%rbx)`, the basic inflation value for the `add` instruction is 1, shown in line 4. The inflation value increases due to the necessity to read and write memory before and after `add` operation, both of which contribute to additional inflation, as shown in lines 7 and 9. The values for basic, memory read, and memory write inflation are extracted by BenchMIAOes, as previously detailed in Section 3.2.
- (3) The *optimizer* simulates the DBT optimizations. Although practical DBTs tend not to apply aggressive optimizations, some conservative optimizations can be applied to translated instructions, such as arithmetic flags elimination [22, 38]. The optimizer does not aim to reproduce the intricate optimization algorithms in DBTs; rather, it functions as a pattern matcher, inspecting instructions within a TB. Upon encountering a specific pattern, it subtracts a specific value from calculated inflation. For example, consider the TB depicted in Figure 9(a), consisting of three instructions: `add`, `cmp`, and `jne`. The inflation calculator calculates the inflation of this TB as $1 + 1 + 2 = 4$. In the example model, when `cmp` is detected adjacent to `jne`, the inflation of `jne` is reduced by 1. The optimizer scans all amenable patterns, as identified by BenchMIAOes, and subtracts the optimized inflation from the overall inflation.

4 EVALUATION

4.1 Experimental Setups

Given the scale of the task and the availability of open source DBTs, we restrict our focus to extracting model parameters from three commercial DBTs using BenchMIAOes, and subsequently construct models for these DBTs using Deflater. Table 2 shows detailed information about the three commercial DBTs. We carried out all experiments on the Linux OS. Thus, we utilized the Linux version of Rosetta2, which runs in just-in-time mode.

Our experiments encompassed not only the industrial standard benchmark (the SPEC CPU 2017 benchmark suite) but also two real-world applications: the widely used interpreter *Python3* and the LLVM-based code prettifier *clang-format*. This demonstrates Deflater's ability to handle both standard benchmarks and real-world workloads. The *Python3* test executes a series of long-run sort algorithms. The *clang-format* test processes a real-world C file comprising approximately 1,000 lines and 3,000 words. We selected these two real-world applications because their executed instructions exhibit a coverage similar to SPEC CPU 2017, as Table 1 illustrates, thereby minimizing additional work. Based on these experimental setups, the rest of this section presents the evaluation of Deflater, including the evaluation of BenchMIAO results and the InflatSim results.

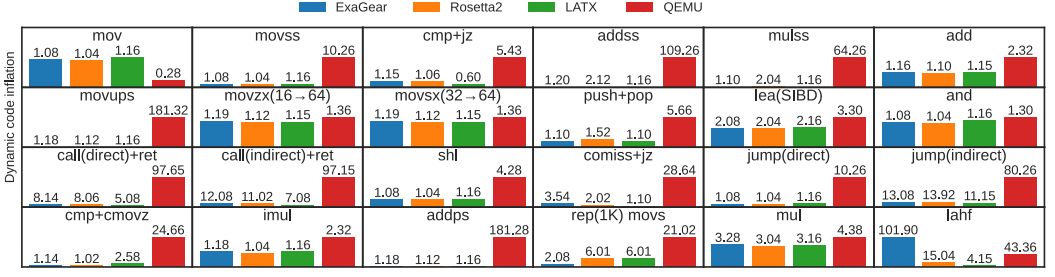


Fig. 10. Dynamic code inflation of representative instruction types extracted by BenchMIAOs from three commercial DBTs.^{4,5}

4.2 BenchMIAO Results: Basic Instruction Types

We have designed more than 200 BenchMIAOs to extract inflation parameters for basic instruction types from DBTs, with a subset of the results depicted in Figure 10. In addition to the three commercial DBTs, we have incorporated the open source DBT, QEMU, to validate the findings of BenchMIAOs. For instance, QEMU’s `mov` instruction has 0 inflation (the .28 decimal stems from the approximation in Equation (2)). This is confirmed by examining its source code since QEMU’s optimization effectively eliminates register move instructions in our BenchMIAOs. Furthermore, QEMU exhibits higher inflation than three commercial DBTs, aligning with Figure 2. Notably, more than half the BenchMIAOs in Figure 10 show an inflation of 1 for the commercial DBTs. The inflation of 1 indicates the similarity of basic instruction types between guest and host ISAs. For instance, x86_64, AArch64, and LoongArch encompass arithmetic instructions like `add` and `cmp(sub)`, shift instructions like `shl`, zero/sign extending instructions like `movzx` and `movsx`, branch instructions like `call/ret` and `jmp/jcc`, and floating-point instructions like `movss` and `addss`. Consequently, these instruction types achieve one-to-one translation. The remaining BenchMIAOs exhibit inflation greater than 1 (decimal values greater than or equal to 2, e.g., 2.xx) or less than 1 (e.g., 0.xx). Subsequent paragraphs analyze the causes behind inflation greater than 1 or less than 1 for three commercial DBTs. For convenience, decimals are rounded down hereafter.

4.2.1 Inflation Greater Than 1. Apart from one-to-one translated instructions, the majority of remaining instructions exhibit inflation greater than 1. The causes of inflation can be broadly categorized into two groups: instruction semantic gap and limitations of the DBT mechanism. *Instruction semantic gap* occurs when the host instruction semantic differs from x86_64 or lacks an equivalent instruction. Here are three representative examples:

- *Arithmetic flags:* X86_64’s arithmetic flags, known as EFLAGS, consist of six flags (SZCOPA). AArch64 has four similar flags (NZCV) but lacks the P and A flags. LoongArch supports flags via EFLAGS emulation instructions [27]. The `lahf` instruction in x86_64 loads five of these flags (excluding the C flag) to the `ah` register. Despite not being used in any SPEC CPU 2017 test, `lahf` can provide insights into how a DBT handles x86_64’s EFLAGS. As illustrated by `lahf`’s inflation in Figure 10, Rosetta2 maximizes the use of AArch64’s flags, achieving low inflation on EFLAGS emulation. High `lahf` inflation suggests that ExaGear may not fully

⁴The inflation of ExaGear’s `lahf` exhibits an unusually large value of 101.90, which is obtained through actual measurements. Intriguingly, this outlier coincides with the inflation of ExaGear’s `cmp+jp` shown later in Table 4. Understanding the reason behind this unusual value requires a comprehensive reverse engineering analysis, which falls beyond the scope of this study.

⁵QEMU uses C helper functions to simulate floating-point instructions, causing high inflation. Importantly, these functions can have varying paths based on inputs (e.g., early termination with 0 multiply), producing different inflation. The inflation presented in the table only represents one category of inputs.

Table 3. Hash Strategy of Three DBTs, Used in Handling Indirect Branch, Extracted by BenchMIAOes

DBT	#Entries	Collision Resolution	Hit Inflation	Miss Inflation
ExaGear	Fixed 512	None	13	~200
Rosetta2	Dynamic	Linear Probing	13	~40
LATX	Fixed 64K	None	11	~500

use flags, possibly emulating EFLAGS lazily. LATX utilizes LoongArch’s EFLAGS emulation instructions, for the lowest lahf inflation.

- Push and pop: AArch64 and LoongArch lack these instructions. However, AArch64 offers pre-indexed and post-indexed load/store instructions. Rosetta2 only utilizes the pre-indexed store and does not utilize the post-indexed load, resulting in extra inflation in pop BenchMIAO. Despite the lack of push/pop-like instructions in LoongArch, LATX achieves nearly one-to-one translation by lazily updating the stack pointer.
- Rep prefix: AArch64 and LoongArch lack an equivalent to the rep prefix, which repeatedly executes an instruction. All three DBTs translate rep movs into a loop. However, ExaGear exhibits lower inflation than both Rosetta2 and LATX, possibly by employing loop unrolling optimization.

Unlike the instruction semantic gap, the inflation from *limitations of DBT mechanisms* is largely independent of the host ISA or translation rules. The most substantial inflation associated with DBT mechanisms arises from indirect branches. Guest and host PC discrepancies require modifying target addresses of branches like jmp, call, and ret to match the host PC for translated code. Direct branch targets can be modified during translation, whereas indirect targets are unknown until runtime. Indirect branches are rare, with about 0.91% in the SPEC CPU 2017 integer. Despite their rarity, indirect branches pose a well-known challenge in DBT research, with inflation higher than 10, as Figure 10 depicts.

Prior studies [12, 16, 37, 45, 58] have proposed various efficient data structures and algorithms to optimize indirect branches. Typically, they utilize a hash table to map guest PCs to translated code. Our analysis reveals that all three commercial DBTs employ a hash table to reduce indirect branch inflation. We extracted the hash table parameters using BenchMIAOes as presented in Table 3. With few jump targets, thus a low hash table miss rate, all three commercial DBTs exhibit low inflation. But with a large number of jump targets, both ExaGear and LATX experience significant inflation, since the fixed hash table causes frequent hash table misses. Rosetta2, benefiting from a dynamic hash table and a linear probing collision resolution, maintains a modest increase in inflation.

4.2.2 Inflation Less Than 1. Although most instructions have inflation equal to or greater than 1, some cases, like cmp+jz shown in Figure 10, have inflation less than 1. cmp+jz represents compare and conditional jump pairs generally. Table 4 presents the inflation for compare and conditional pairs in three commercial DBTs.

Two AArch64-based DBTs achieve mostly one-to-one translation, owing to similar semantics of conditional jumps in x86_64 and AArch64. The x86_64 has eight types of condition codes, each with corresponding negative versions, whereas AArch64 has seven types with corresponding negative versions. Due to the absence of parity conditional code in AArch64, Rosetta2 and ExaGear emulate it, resulting in 3.5 and 42 inflation, respectively.

LATX fuses more than half of the conditionals into two-to-one translation, as LoongArch uses compare-then-jump instructions. LoongArch has three types of conditional jumps: beq, bltu, and bge, each with its corresponding negative versions. LATX leverages these conditional jumps,

Table 4. Compare and Eight Types of Conditional Jumps Translation and Inflation

X86 Instruction X86 Conditional Code	je, jz Z	jo O	jc, jb, jnae C	ja, jnbe !C&!Z	js S	jp, jpe P	jge, jnl S==O	jng, jle S!=O Z
AArch64 Instruction AArch64 Conditional Code	b.eq Z	b.vs V	b.cs, b.hs C	b.hi C&!Z	b.mi N	– –	b.ge N==V	b.le N!=V Z
Rosetta2 Inflation	1	1	1	1	1	3.5	1	1
ExaGear Inflation	1	1	1	1	1	42	1	1
LoongArch Instruction LATX Inflation	beq 0.5	– 1	bltu 0.5	bltu* 0.5	– 1	– 1	bge 0.5	bge* 0.5

The negative versions of these eight conditional jumps are omitted, as their inflation is the same as the corresponding positive one.⁶

enabling the fusion of `cmp+je`, `cmp+jc`, and `cmp+jge` into a single LoongArch instruction. Furthermore, by swapping the two compared operands, LATX can fuse `cmp+ja` and `cmp+jng` into `bltu` and `bge`, respectively. Although LoongArch lacks similar instructions to `jo`, `js`, and `jp`, LATX translates them using LoongArch’s EFLAGS emulation extension, resulting in the inflation of 1 in these BenchMIAOs.

4.3 BenchMIAO Results: Instruction Variants

This subsection presents and analyzes instruction variant BenchMIAO results for the three commercial DBTs in four categories: memory accessing, addressing, sub-register, and immediate loading.

4.3.1 Memory Accessing. BenchMIAO results reveal that all three commercial DBTs translate load and store variants into one additional host instruction. For example, a basic instruction, `add %rax, %rbx`, is typically translated into one host `add` instruction, whereas its memory accessing variant, `add (%rax), %rbx`, is translated into two host instructions, where one additional load instruction is necessary to access the memory pointed to by `rax`. This finding suggests that all three DBTs linearly map GVA to HVA with `guest_base = 0`, as depicted in the right half of Figure 8.

4.3.2 Addressing. X86_64 has 11 valid combinations of scale, index, base, and displacement for addressing modes. X86_64 also supports PC-related addressing, like PC+displacement. Table 5 presents the results for these 12 types of addressing modes. Theoretically, disregarding immediate variants, all addressing modes could be translated into two host instructions. This is possible because AArch64’s `add.lsl` and LoongArch’s `alsl` perform shift and addition in a single instruction. Moreover, both AArch64 and LoongArch support load/store instructions with an immediate offset. For instance, `SIBD` can be translated into `alsl tmp, B, I, S` and `ld dst, tmp, D`. Additionally, in ExaGear and Rosetta2, the `IB` and `IBD` are translated into a single instruction due to AArch64’s load-store instruction supporting a register offset. Table 5 also demonstrates the potential optimizations for addressing modes. For instance, the `ID`, `IBD`, `ID`, and `I` addressing modes could achieve inflation of 1 in both LoongArch and AArch64.

4.3.3 Sub-Register. Table 6 presents the results of sub-register BenchMIAOs. In x86_64, when using 8-bit or 16-bit sub-registers, the higher bits remain unmodified. Consequently, bit extraction

⁶The inflation of ExaGear’s `cmp+jp` exhibits an unusually large value of 42, which is obtained through actual measurements. This inflation indicates that `cmp+jp` are translated into around 84 host instructions. Intriguingly, this outlier coincides with the inflation of ExaGear’s `lahf` shown in Figure 10. Understanding the reason behind this unusual value requires a comprehensive reverse engineering analysis, which falls beyond the scope of this study.

Table 5. Inflation of Addressing Modes (mov)

DBT	SI	SID	B	BD	SIB	SIBD	IB	IBD	ID	I	D	PD
ExaGear	2	3	1	1	2	3	1	1	2	1	1	1
Rosetta2	2	3	1	1	2	2	1	1	1	1	1	1
LATX	2	2	1	1	2	2	2	2	2	2	1	1

S, scale; I, index; B, base; D, displacement; P, PC.

Table 6. Inflation of Sub-Registers (add)

DBT	l→l	h→l	l→h	h→h	16	32
ExaGear	2	3	3	4	2	1
Rosetta2	2	3	3	4	2	1
LATX	2	3	3	4	2	2

l, low 8-bit register; h, high 8-bit register.

Table 7. Inflation of Sub-Registers for Extension Instructions (movsx, movzx)

Source	l	h	l	h	16	16	32	l	h	l	h	l	16	16
Sign/Zero	s	s	s	s	s	s	s	z	z	z	z	z	z	z
Destination	16	16	32	32	32	64	64	16	16	32	32	64	32	64
ExaGear	2	3	1	2	1	1	1	2	3	1	2	1	1	1
Rosetta2	2	2	1	1	1	1	1	2	2	1	1	1	1	1
LATX	2	3	2	3	2	2	1	2	2	1	1	1	1	1

l, low 8-bit register; h, high 8-bit register.

Table 8. Inflation of Three Types of Immediate Loading (mov)

DBT	Effective Length								Hole				Bitmask	
	12	16	24	32	36	48	52	64	[15:0]	[11:0]	[31:16]	[32:12]	simple	complex
ExaGear	1	1	2	2	3	3	4	4	3	4	3	3	1	4
Rosetta2	1	1	2	2	3	3	4	4	3	4	3	3	1	4
LATX	1	2	2	2	3	3	3	4	3	3	4	3	4	4

and insertion instructions are utilized to translate sub-registers. The same inflation is observed in the 8-bit and 16-bit BenchMIAOes across all three DBTs. In x86_64 and AArch64, when the destination register is 32-bit, the value is zero extended to the higher bits ([63:32]). However, in LoongArch, the higher bits are sign extended, resulting in additional inflation in 32-bit BenchMIAOes.

In addition to using sub-registers of identical sizes, move extension instructions employ sub-registers of different sizes. Table 7 presents the results of the move extension inflation. Among the three commercial DBTs, Rosetta2 exhibits the lowest inflation, as AArch64 has `sbfx` and `ubfx`, which can sign/zero extend any bitfield. ExaGear employs AArch64's basic extension instructions, such as `sxtb` and `uxtb`, which require cooperation with bit extraction and insertion instructions, leading to slightly higher inflation compared with Rosetta2. Since LoongArch sign extends the higher 32 bits, LATX requires additional instructions when sign extending to the 32-bit sub-register.

4.3.4 Immediate Loading. The immediate loading BenchMIAOes are designed by utilizing x86_64's `mov` instruction, which allows direct encoding of a 64-bit immediate. To evaluate the immediate loading inflation in different scenarios, we design the immediate loading BenchMIAOes using three methods: effective length, hole, and bitmask. The subsequent paragraphs present and analyze the results of these three types of BenchMIAOes.

The *effective length* refers to the actual length of an immediate number, disregarding any leading zeros or ones (sign bits). For instance, considering a 64-bit immediate `0xffffffff6bcdef0123`, possessing 24 leading ones, has an effective length of 40 bits. Zero/sign extension instructions can cooperate with immediate patching instructions, eliminating the requirement for full-width patching of long immediates. Table 8 reveals that AArch64 employs `mov` and `movk` in 16-bit lengths, and LoongArch uses `ori`, `lu12i`, `lu32i`, and `lu52i` for 64-bit patching in 12/20/20/12-bit lengths.

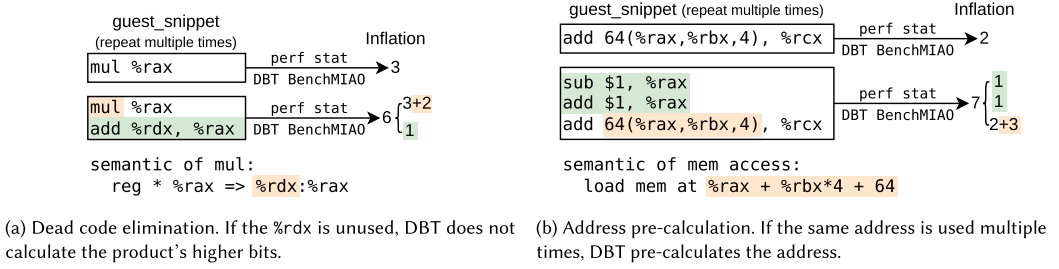


Fig. 11. Examples of identifying optimizations in DBTs by BenchMIAOes.

The *hole* BenchMIAOes aim to verify whether DBTs can reduce the number of patching instructions when the patch target comprises multiple consecutive zeros or ones in the middle. For example, the [31:16] bits of `0xabfabc110000a304` are all zeros, hence three AArch64 patching instructions suffice to generate this immediate. We refer to this phenomenon as a patch-up hole. The inflation associated with hole BenchMIAOes, as shown in Table 8, confirms that all three DBTs can effectively handle patch-up holes.

The *bitmask* BenchMIAOes primarily determine whether AArch64-based DBTs can utilize AArch64's bitmask instruction to load immediates. The BenchMIAO results reveal that both ExaGear and Rosetta2 can manage simple bitmask immediate, such as `0x3333333333333333`, using AArch64's bitmask move instruction. However, neither ExaGear nor Rosetta2 is able to handle complex bitmask immediates, such as `0x4CCC33334CCC3333`. This complex bitmask immediate can be loaded using two AArch64 instructions: first loading `0x3333333333333333` and then executing a bitmask xor instruction with `0x7FFF00007FFF0000`.

4.4 BenchMIAO Results: Optimizations

As stated in Observation 1, practical DBTs tend to avoid aggressive optimizations to maintain the guest's precise exceptions. However, BenchMIAOes empirically identify conservative optimizations present in three commercial DBTs. For example, as illustrated in Figure 11(a), the `x86_64 mul` instruction performs an unsigned multiplication of `%rax` with another register and stores the product in `%rdx:%rax`. If the higher 64 bits of the product remain unused, the measured inflation is 3. However, if both the higher and lower bits are used, the inflation increases by 3, exceeding the inflation of an add instruction. This phenomenon suggests that the DBT eliminates the calculation of the unused higher bits, resulting in the reduction of inflation by 2. Another example, as depicted in Figure 11(b), involves the repeatedly executed add instruction, which adds a memory operand to a register operand, yielding inflation of 2. If we modify the base register, the overall inflation rises to 7, surpassing the inflation associated with the add and sub instructions. This phenomenon suggests that the DBT pre-calculates the memory addressing `64(%rax,%rbx,4)`, allowing the inflation to be reduced to 2, which includes a memory load and an addition. Table 9 summarizes the potential optimizations identified by the BenchMIAOes, along with their explanations.

The identified optimizations can be categorized into two groups: ISA independent and ISA dependent. The ISA-independent optimizations listed in Table 9 include dead code elimination, immediate loading pre-calculation, address pre-calculation, and loop unrolling. Our analysis reveals that none of the commercial DBTs utilize all of these optimizations. Experimental results from our in-house DBT show that these optimizations not only decrease inflation but also lead to reduced execution time. The remaining optimizations listed in Table 9 are ISA dependent. Contiguous memory access fusion requires the support of load/store pair instructions, which are uncommon in

Table 9. Identified Optimizations in Three Commercial DBTs

Optimization	Descriptions
Dead code elimination	For all three commercial DBTs, redundant EFLAGS is eliminated. Additionally, in Rosetta2, the unused higher bits of mul instruction are discarded. These dead code elimination optimizations are typically conducted across TBs. As explained in Equation (1), we simplify these elimination optimizations by assuming all dead code can be eliminated. Therefore, these optimizations are not simulated in the following InflatSim simulation, for the tradeoff between model complexity and accuracy.
Imm loading pre-calculation	To prevent redundant loading of the same immediate, the loaded immediate value can be pre-calculated and saved in a temporary register. Our analysis reveals that none of the three commercial DBTs have incorporated this optimization.
Address pre-calculation	When multiple loads/stores within a TB share the same addressing mode and the registers associated with the addressing mode remain unmodified between these loads/stores, ExaGear will compute the memory address before these instructions and store it in a temporary register. This optimization allows subsequent loads/stores to retrieve the memory address without recalculating it.
Contiguous mem access fusion	If multiple loads/stores access a contiguous memory space, ExaGear employs ldp/stp to load/store a pair of data. Push/pop pairs are the most common optimizations utilizing ldp/stp in ExaGear and Rosetta2. Furthermore, ExaGear combines floating-point and vector contiguous loads/stores, like movss and movpsd.
Push/pop elision	If the guest program only modifies the stack pointer using push/pop instructions within a TB, LATX optimizes the stack operations by combining the decrease and increase of the stack pointer and updating the stack pointer only at the TB exit. This optimization addresses the lack of stack instructions in LoongArch.
Loop unrolling	In DBTs, the Rep prefix is translated into a loop. ExaGear tends to unroll the translated loop to achieve lower inflation.
Cmp and jcc fusion	As demonstrated in Section 4.2.2, LATX merges cmp and 10 out of 16 types of jccs into a single conditional jump.

RISC ISAs. Push/pop elision is useful in RISC ISAs, such as MIPS, LoongArch, and RISC-V, because these ISAs typically archive stack operations using store/load and sub/add instructions. Likewise, compare and conditional jump fusion is beneficial for RISC ISAs that typically support compare-then-jump instructions.

4.5 InflatSim Results

Leveraging the model parameters extracted through BenchMIAOes, we model ExaGear, Rosetta2, and LATX using Equation (1) and instantiate it in InflatSim. Figure 12 depicts the simulated inflation. Overall, InflatSim exhibits an average inflation error of 5.63%, 5.15%, and 3.44% for ExaGear, Rosetta2, and LATX, respectively, calculated using Equation (4). This inflation error primarily stems from two factors: inflation from unidentified instruction and unidentified optimizations. The Pearson correlation coefficient between real and simulated inflation is 89.77%, 89.14%, and 95.41% for ExaGear, Rosetta2, and LATX, respectively. A correlation coefficient exceeding 80% is generally considered indicative of a strong correlation between two datasets.

$$inflation_error = \frac{|real_inflation - simulated_inflation|}{real_inflation} \quad (4)$$

Figure 12 also shows the primary sources of inflation. Memory access, immediate loading, and addressing variants constitute the majority of the inflation. The sub-register variant contributes

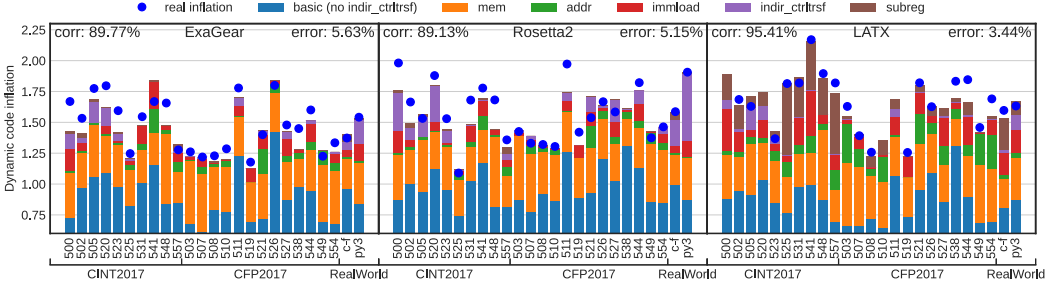


Fig. 12. Real (dots) and decomposed simulated (bars) inflation for SPEC CPU 17. Higher correlation and lower error are better.

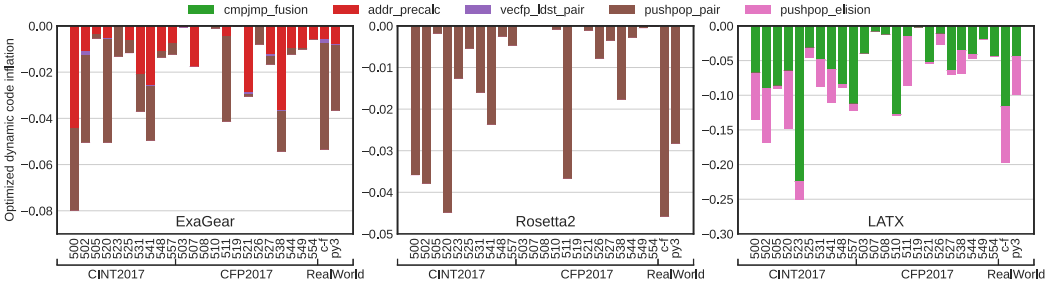


Fig. 13. Simulated inflation reduction by optimizations, representing the subtraction in Equation (1). (SPEC CPU 17, ref input set).

minimally to inflation in AArch64 since most sub-registers are 32 bits. In contrast, the sub-register variant exhibits high inflation in LATX, as the LoongArch instructions sign extends the 32-bit destination register to 64 bits. In addition to the inflation from these variants, indirect branches also incur significant inflation. However, they account for approximately 0.91% of SPEC CPU 17. This stems from the considerable overhead of hash table queries. The remaining inflation originates from basic instructions. Since the basic instruction types are too numerous to display individually in Figure 12, they are aggregated in the blue bars. For the ratio and inflation of basic instruction types, please refer to Table 1 and Figure 10, respectively. Overall, LATX achieves lower inflation in basic instructions compared with ExaGear and Rosetta2, implying effective optimizations of basic instructions.

Figure 9 shows the inflation reduction achieved by the optimizations identified in Figure 13. In ExaGear, the most noticeable optimizations are address pre-calculation, vector load/store pair fusion, and contiguous push/pop pair fusion. These yield a maximum dynamic inflation reduction of 0.08. In Rosetta2, only the contiguous push/pop pair fusion reduces noticeable inflation, contributing to less than 0.05 inflation reduction in dynamic code inflation. LATX achieves substantial inflation reduction through its compare and conditional jump fusion and push/pop elision, with multiple tests exhibiting greater than 0.10 inflation reduction.

4.6 Insights into the Results

The insights we gained from the simulation results of three commercial DBTs using Deflater are presented in Tables 10–12, divided into three categories of ISA features, translation rules, and optimizations, respectively.

Table 10. ISA Features Identified in AArch64 and LoongArch That Are Conducive to Translating x86_64 Instructions

ISA Feature	Insights
Zero extension	AArch64 zero extends the 32-bit sub-register, which matches the behavior of x86_64, thus achieving low inflation on x86_64 32-bit sub-register translation.
EFLAGS emulation	LoongArch and AArch64 have distinct strengths in translating x86_64's EFLAGS. LoongArch utilizes its EFLAGS emulation instructions, resulting in an extra flags calculation instruction. AArch64's arithmetic flags are similar to those of x86_64 but lack the P flag, resulting in high inflation on translating the P flag.
Conditional jump	LoongArch utilizes its compare-then-jump instructions, achieving two-to-one translation on several types of compare-conditional-jump pairs.

Table 11. Optimal Translation Rules and Their Translation Inflation in AArch64 and LoongArch

Translation Rule	Insights
Indirect branch	The combination of LATX's translation rule with Rosetta2's dynamic hash table is able to achieve optimal performance.
Other basic instructions	The efficient translation inflation is demonstrated in Figure 10.
Mem accessing variant	The guest base is set to zero so that only one extra load/store instruction is needed.
Addressing variant	The combination of shifted addition and offset load/store achieves low inflation.
Sub-register variant	Relying on the 32-bit zero extension ISA feature, the sub-register variant does not incur noticeable inflation as simulation results show.
Imm loading variant	The combination of immediate patch-up and bitmask method achieves low inflation.

Table 12. Identified Optimizations and Possible Optimization Research in the Future

Optimization	Insights
Imm loading pre-calculation	This potential optimization is not identified in three DBTs.
Address pre-calculation	The use of complex addressing mode is common in x86_64, thus this optimization reduces noticeable inflation, as shown in Figure 13.
Contiguous mem access fusion	Although vector and float-point load/store instruction fusion and push/pop fusion optimizations are identified in AArch64-based DBTs, they rely on load/store pair instructions, which are uncommon in RISC-style ISAs.
Push/pop elision	Figure 13 demonstrates that LATX's push/pop elision optimization noticeably reduces inflation, addressing the lack of push/pop in LoongArch.
Loop unrolling	Since the occurrence of the rep prefix in SPEC CPU 2017 is rare, the inflation reduced by this optimization is negligible.
Cmp and jcc fusion	LoongArch supports fusing part of the compare and conditional jump pairs, as shown in Table 4. The inflation reduced by this optimization surpasses that of all other identified optimizations. Therefore, it will be valuable to investigate additional fusible instruction pairs in future research.

5 CASE STUDY: THE APPLICATION OF DEFLATER

To showcase Deflater's capabilities, we have integrated it into our real-world development workflow to facilitate the optimization of the open source DBT, *QEMU*. Deflater efficiently constructs an inflation model and provides essential insights into potential inflation reduction. Thus, Deflater can save developers valuable time before they embark on typically time-consuming optimization efforts.

The left side of Figure 14 illustrates a typical DBT optimization workflow. Since the translated code is dynamically generated, conventional performance analysis tools such as *perf* can identify the hot translated code but cannot determine its source. Consequently, DBT developers usually devise potential optimizations based on their experience. Implementing these optimizations can then take substantial time, potentially several weeks, for the DBT developer. With luck, the DBT

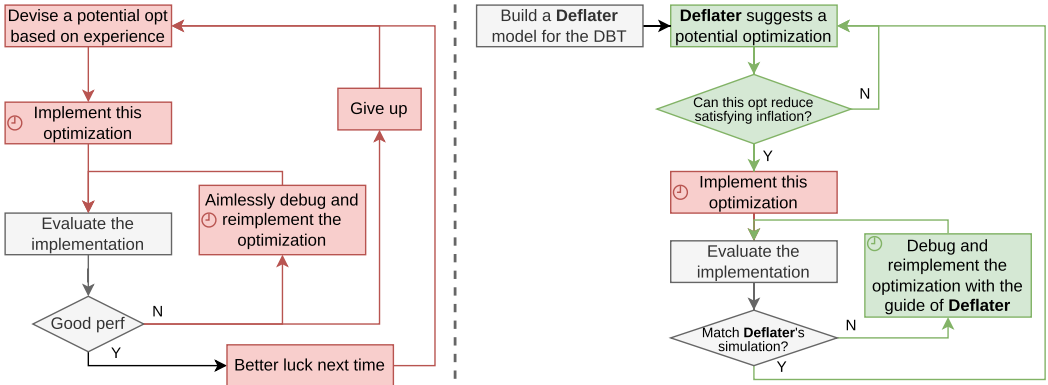


Fig. 14. The DBT optimization workflows without the guide of Deflater (left) and with the guide of Deflater (right).

developer may succeed in improving performance on the initial attempt. However, without proper performance analysis tools, debugging and reimplementing complex DBT optimizations can be an unguided, time-consuming process—potentially lasting weeks or months.

Nevertheless, using Deflater can streamline the DBT optimization workflow, as depicted on the right side of Figure 14. To demonstrate Deflater’s optimization guidance, we optimized QEMU, with RISC-V as the guest architecture and LoongArch as the host. We found that compared with the native execution, QEMU running SPEC CPU 2017 suffered more than 10x slowdown and 15x dynamic instruction inflation. To optimize it, we followed the workflow presented next.

First, we designed a series of RISC-V BenchMIAOes, extracted model parameters from QEMU, and constructed a DeflatSim model for QEMU in days. SPEC CPU 2017 results reveal actual inflations of 9.87 for CINT and 28.50 for CFP, whereas the simulated inflations are 9.97 for CINT and 28.63 for CFP with a 6.61% inflation error.

Second, the DeflatSim model indicates that QEMU experiences significant inflation in nearly all instruction translations. This is primarily due to QEMU’s use of a two-stage translation process to accommodate multiple ISAs, wherein guest instructions are initially converted to an intermediate representation known as TCG. Subsequently, TCG instructions are translated into host instructions. If TCG lacks support for specific guest instructions, such as floating-point instructions, it resorts to invoking C-written functions to simulate guest semantics, resulting in notable inflation. Hence, transitioning from this two-stage translation to a direct guest-to-host translation could be a viable optimization.

Third, we assessed the effectiveness of this optimization by modifying the inflation parameters of the DeflatSim model, a task accomplished within hours and about 400 lines of code edited. The simulation results demonstrate that the inflation rates for CINT and CFP decrease substantially by 84.2% and 94.4%, resulting in values of 1.58 and 1.60, respectively, which is highly appealing.

Fourth, we eliminated the TCG IR representation, achieving the end-to-end translation from RISC-V to LoongArch. During this process, we also realized two optimizations mentioned earlier: compare and conditional jump fusion, register mapping. The implementation of this optimization required several weeks and the editing of about 8,000 lines of code. Experimental results reveal that our implementation results in inflation values of 1.62 and 1.55 for CINT and CFP, respectively, with a 4.65% inflation error. Notably, the optimized QEMU achieved an overall 5.47x performance improvement, with 2.99x for CINT and 7.12x for CFP.

In summary, we modeled QEMU using Deflater and identified multiple feasible optimization approaches. We rapidly assessed the optimization impact using Deflater and then proceeded with

specific implementations. Deflater reduced the trial-and-error costs, expediting the implementation of QEMU optimization algorithms.

6 RELATED WORK

This section provides an overview of the related works on DBT overhead analysis. We categorize these related works into two groups: overall overhead analysis and specific overhead analysis.

6.1 Overall Overhead Analysis for DBTs

Numerous previous studies have examined the coarse-grained overhead in DBT. For instance, Nimmakayala [43] and Rodríguez et al. [50] investigate the overhead in same-ISA DBT through benchmark-based evaluations. Borin and Wu [11] analyze the DBT overhead in five components: initialization, cold code translation, code profiling, hot trace building, and translated code execution. Moore et al. [42] and Ruiz-Alvarez and Hazelwood [51] concentrate on cache and TLB impacts using hardware performance monitors. Martins do Rosário et al. [20] develop a DBT simulator evaluating various region formation algorithms.

Since this article focuses on user-level DBTs, few benchmarks target them specifically. Nevertheless, there are benchmarks designed for the full system/virtualization, like SimBench [59], VITS Test Suit [68], and HyperBench [66]. For instance, SimBench aggregates results from micro-benchmarks to create an estimator for overall application performance, a concept akin to BenchMIAOes.

Aside from the analysis studies, several studies focus on generating more efficient translated code. ISAMAP [55] and Captive [56] generate translation rules from high-level instruction descriptions. A series of studies [30, 54, 61] learn translation rules by compiling the same source code to guest and host ISAs.

6.2 Specific Overhead Analysis for DBTs

Various techniques have addressed indirect branch overhead. For instance, DynamoRIO [12], Pin [37], HDTrans [58], and FastBT [45] utilize prediction to minimize overhead in the same ISA. MamBox64 [16] consolidates and incorporates state-of-the-art indirect branch optimizations for ARM32 to ARM64 translation. Lazy evaluation is utilized by Ma et al. [38] and EfLA [22] to prune redundant arithmetic flag calculations, thus improving arithmetic flag performance. Additionally, Harmonia [44] and Wang et al. [61] maximize the mapping between guest and host arithmetic flags.

In addition to the software-based works, hardware is utilized to reduce the overhead of translated code. Dedicated hardware [32, 34, 52] is employed to accelerate indirect branch target lookups. The integration of DBT and VLIW processors has been investigated in various systems, including Transmeta's Crusoe [33], IBM's DAISY [21], and NVIDIA's Denver [9], to leverage **Instruction-Level Parallelism (ILP)** with the help of DBT. Instruction fusion is a well-established hardware optimization used in current high-performance CPUs, including certain x86_64 CPUs [15] and AArch64 CPUs [4, 5]. The possibilities of software-based instruction fusion are explored by Hu and Smith [26] and SoftHV [19] to improve performance on dedicated hardware. ISA extensions serve as a means to bridge the semantic gap between guest and host ISA. For instance, Loongson's LATX leverages the DBT extension of LoongISA [27] and LoongArch ISA [36]. Additionally, RISC-V [65] provides the B extension for bit manipulation, the J extension for dynamically translated languages, and the P extension for packed-SIMD instructions.

7 DISCUSSION AND FUTURE WORK

Deflater is limited to simulating DBTs that preserve instruction boundaries. This limitation arises from Deflater's mathematical model, which is grounded in Observation 1 that real DBT products

typically refrain from breaking instruction boundaries to ensure the guest's precise exception handling. However, an increasing amount of research explores DBTs that use aggressive optimizers like LLVM, disregarding precise exception handling to enable more optimizations. Analyzing the overhead of these highly optimized DBTs requires a new approach, such as developing a statistical model, which can be investigated in future work.

Our analysis reveals that the primary overhead in computational workloads lies within the translated code. Hence, Deflater proves to be well suited for characterizing this computational workload overhead. However, DBT workloads vary, with performance affected by other overhead factors beyond the translated code. For instance, workloads with poor code locality may experience performance degradation due to frequent code generation. The performance of large codebases may be impacted by software code cache management policies.

Moreover, Deflater does not model the hardware overhead. With multi-threaded workloads, if the guest employs a stronger memory model (e.g., x86's TSO memory model) than the host (e.g., ARM's weak memory model), DBT must emit fence instructions for correctness. Prior research [24, 49] focuses on efficient and accurate inter-thread memory access translation. Additionally, translating from a guest with a coherent L1 instruction cache (e.g., x86) to a non-coherent one (e.g., ARM) requires careful handling of dynamically generated code [31, 62]. Currently, Deflater lacks BenchMIAOes tailored for identifying inflation from multi-threading and dynamically generated code, as these aspects are not typically found in computational workloads like SPEC CPU benchmarks. Nevertheless, both multi-threading and dynamically generated code represent significant DBT research areas and will be subject to future analysis.

While BenchMIAOes successfully identified DBT optimizations, this currently relies on empirical evidence. Unlike BenchMIAOes for various instruction types and variants, which leverage dynamic instruction frequency (as depicted in Table 1) and ISA semantic, identifying potential optimizations involves practical experience and a trial-and-error approach. Furthermore, interpreting the measured inflation necessitates DBT expertise, as Section 4.4 demonstrates. Future work will investigate automating BenchMIAOes creation to facilitate optimization identifications.

8 CONCLUSION

To gain insights into translation inflation in DBTs, we presented *Deflater*, an inflation analyzing framework. Deflater consists of three components: a mathematical model for calculating DBT inflation, a collection of black-box unit tests named *BenchMIAOes*, and an InflatSim simulator that creates models for DBT inflation. By utilizing Deflater, we analyzed three commercial x86_64-to-RISC DBTs—ExaGear, Rosetta2, and LATX, exhibiting low inflation errors of 5.63%, 5.15%, and 3.44%, respectively. Our experimental results also revealed that the primary sources of inflation are memory access, immediate load, address calculation, sub-register access, and indirect branch. Moreover, we employed Deflater in a practical development process to optimize the open source DBT, *QEMU*. Deflater efficiently simulated *QEMU*'s dynamic instruction inflation and suggested optimizations that can significantly reduce inflation with a 4.65% inflation error. Our implementation of the suggested optimizations validates the effectiveness of Deflater's guidance, resulting in approximately 90% reduction in inflation and a 5.47x performance improvement.

REFERENCES

- [1] Apple. 2021. About the Rosetta Translation Environment. Retrieved March 3, 2023 from <https://developer.apple.com/documentation/apple-silicon/about-the-rosetta-translation-environment>
- [2] Apple. 2022. Running Intel Binaries in Linux VMs with Rosetta. Retrieved March 30, 2023 from https://developer.apple.com/documentation/virtualization/running_intel_binaries_in_linux_vms_with_rosetta
- [3] ARM. 2013. *ARM CoreSight Architecture Specification v2.0*. ARM.
- [4] ARM. 2019. *Cortex®-A77 Software Optimization Guide*. ARM.

- [5] ARM. 2020. *Arm® Neoverse™ N2 Software Optimization Guide*. ARM.
- [6] Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. 2000. Dynamo: A transparent dynamic optimization system. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation*. 1–12.
- [7] Leonid Baraz, Tevi Devor, Orna Etzion, Shalom Goldenberg, Alex Skaletsky, Yun Wang, and Yigal Zemach. 2003. IA-32 execution layer: A two-phase dynamic translator designed to support IA-32 applications on Itanium/spl reg/-based systems. In *Proceedings of the 2003 36th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '03)*. IEEE, 191–201.
- [8] Fabrice Bellard. 2005. QEMU, a fast and portable dynamic translator. In *Proceedings of the USENIX Annual Technical Conference: FREENIX Track*, Vol. 41. 46.
- [9] Darrell Boggs, Gary Brown, Nathan Tuck, and K. S. Venkatraman. 2015. Denver: NVIDIA's first 64-bit ARM processor. *IEEE Micro* 35, 2 (2015), 46–55.
- [10] Igor Böhm, Tobias J. K. Edler von Koch, Stephen C. Kyle, Björn Franke, and Nigel Topham. 2011. Generalized just-in-time trace compilation using a parallel task farm in a dynamic binary translator. *ACM SIGPLAN Notices* 46, 6 (2011), 74–85.
- [11] Edson Borin and Youfeng Wu. 2009. Characterization of DBT overhead. In *Proceedings of the 2009 IEEE International Symposium on Workload Characterization (IISWC '09)*. IEEE, 178–187.
- [12] Derek Bruening and Saman Amarasinghe. 2004. *Efficient, Transparent, and Comprehensive Runtime Code Manipulation*. Ph.D. Dissertation. Massachusetts Institute of Technology.
- [13] GitHub. 2013. Capstone-Engine: Capstone Disassembly/Disassembler Framework. Retrieved March 30, 2023 from <https://github.com/capstone-engine/capstone>
- [14] Anton Chernoff, Mark Herdeg, Ray Hookway, Chris Reeve, Norman Rubin, Tony Tye, S. Bharadwaj Yadavalli, and John Yates. 1998. FX! 32: A profile-directed binary translator. *IEEE Micro* 18, 2 (1998), 56–64.
- [15] Intel Corporation. 2022. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Intel.
- [16] Amanieu d'Antras, Cosmin Gorgovan, Jim Garside, and Mikel Luján. 2016. Optimizing indirect branches in dynamic binary translators. *ACM Transactions on Architecture and Code Optimization* 13, 1 (2016), 1–25.
- [17] Amanieu d'Antras, Cosmin Gorgovan, Jim Garside, and Mikel Luján. 2017. Low overhead dynamic binary translation on arm. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 333–346.
- [18] Arnaldo Carvalho De Melo. 2010. The new Linux 'perf' tools. In *Slides from Linux Kongress*, Vol. 18. 1–42.
- [19] Abhishek Deb, Josep Maria Codina, and Antonio González. 2011. SoftHV: A HW/SW co-designed processor with horizontal and vertical fusion. In *Proceedings of the 8th ACM International Conference on Computing Frontiers*. 1–10.
- [20] Vanderson Martins do Rosário, Raphael Zinsly, Sandro Rigo, and Edson Borin. 2021. Employing simulation to facilitate the design of dynamic binary translators. In *Proceedings of the 2021 IEEE 33rd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD '21)*. 104–113.
- [21] Kemal Ebcioglu, Erik Altman, Michael Gschwind, and Sumedh Sathaye. 2001. Dynamic binary translation and optimization. *IEEE Transactions on Computers* 50, 6 (2001), 529–548.
- [22] Feng Tang, Cheng-Gang Wu, Xiao-Bing Feng, and Zhao-Qing Zhang. 2007. EflA algorithm based on dynamic feedback. *Journal of Software* 18, 7 (2007), 1603–1611.
- [23] GitHub. 2018. FEX-Emu: A Fast Usermode x86 and x86-64 Emulator for ARM64. Retrieved March 30, 2023 from <https://github.com/FEX-Emu/FEX>
- [24] Redha Gouicem, Dennis Sprockholt, Jasper Ruehl, Rodrigo C. O. Rocha, Tom Spink, Soham Chakraborty, and Pramod Bhatotia. 2022. Risotto: A dynamic binary translator for weak memory model architectures. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Vol. 1. 107–122.
- [25] Ding-Yong Hong, Chun-Chen Hsu, Pen-Chung Yew, Jan-Jan Wu, Wei-Chung Hsu, Pangfeng Liu, Chien-Min Wang, and Yeh-Ching Chung. 2012. HQEMU: A multi-threaded and retargetable dynamic binary translator on multicores. In *Proceedings of the 10th International Symposium on Code Generation and Optimization*. 104–113.
- [26] Shiliang Hu and James E. Smith. 2004. Using dynamic binary translation to fuse dependent instructions. In *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO '04)*. IEEE, 213–224.
- [27] Weiwu Hu, Jian Wang, Xiang Gao, Yunji Chen, Qi Liu, and Guojie Li. 2009. Godson-3: A scalable multicore RISC processor with x86 emulation. *IEEE Micro* 29, 2 (2009), 17–29.
- [28] Huawei. 2022. Huawei Kunpeng ExaGear. Retrieved March 30, 2023 from <https://mirrors.huaweicloud.com/kunpeng/archive/ExaGear/>
- [29] Intel. 2018. Processor trace. In *Intel® 64 and IA-32 Architectures Software Developer's Manual*. Vol. 3. Intel, 4025–4104.
- [30] Jinhu Jiang, Rongchao Dong, Zhongjun Zhou, Changheng Song, Wenwen Wang, Pen-Chung Yew, and Weihua Zhang. 2020. More with less—Deriving more translation rules with less training data for DBTs using parameterization. In *Proceedings of the 2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '20)*. IEEE, 415–426.

- [31] David Keppel. 2009. How to detect self-modifying code during instruction-set simulation. In *Proceedings of the IEEE/ACM Workshop on Architectural and Microarchitectural Support for Binary Translation*.
- [32] H.-S. Kim and James E. Smith. 2003. Hardware support for control transfers in code caches. In *Proceedings of the 2003 36th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '03)*. IEEE, 253–264.
- [33] Alexander Klaiber. 2000. *The Technology Behind Crusoe Processors: Low-Power x86-Compatible Processors Implemented with Code Morphing Software*. Technical Brief. Transmeta Corporation.
- [34] Tingtao Li, Alei Liang, Bo Liu, Ling Lin, and Haibing Guan. 2008. A hardware/software codesigned virtual machine to support multiple ISAs. In *Proceedings of the AMSBT Conference*. 38–44.
- [35] Yi Liang, Yuanhua Shao, Guowu Yang, and Jinzhao Wu. 2015. Register allocation for QEMU dynamic binary translation systems. *International Journal of Hybrid Information Technology* 8, 2 (2015), 199–210.
- [36] China Loongson Technology. 2023. *LoongArch Reference Manual—Volume 3: Virtualization and Binary Translation Extensions*. Loongson Technology Corporation Ltd.
- [37] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: Building customized program analysis tools with dynamic instrumentation. *ACM SIGPLAN Notices* 40, 6 (2005), 190–200.
- [38] Xiangning Ma, Chenggang Wu, Feng Tang, Xiaobing Feng, and Zhaoqing Zhang. 2005. Two condition code optimization approaches in binary translation. *Jisuanji Yanjiu yu Fazhan (Computer Research and Development)* 42, 2 (2005), 329–337.
- [39] Microsoft. 2023. Windows on Arm Documentation. Retrieved March 30, 2023 from <https://learn.microsoft.com/en-us/windows/arm/overview>
- [40] GitHub. 2023. MicroTranslator/BenchMIAO: A Collection of Black-Box Unit Tests for DBT Inflation Analysis. Retrieved January 18, 2024 from <https://github.com/MicroTranslator/BenchMIAO>
- [41] GitHub. 2023. MicroTranslator/InflatSim: A Trace-Based DBT Inflation Simulator. Retrieved January 18, 2024 from <https://github.com/MicroTranslator/InflatSim>
- [42] Ryan W. Moore, José A. Baiocchi, Bruce R. Childers, Jack W. Davidson, and Jason D. Hiser. 2009. Addressing the challenges of DBT for the ARM architecture. *ACM SIGPLAN Notices* 44, 7 (2009), 147–156.
- [43] Surya Tej Nimmakayala. 2015. *Exploring Causes of Performance Overhead during Dynamic Binary Translation*. Ph.D. Dissertation. University of Kansas.
- [44] Guilherme Ottoni, Thomas Hartin, Christopher Weaver, Jason Brandt, Belliappa Kuttanna, and Hong Wang. 2011. Harmonia: A transparent, efficient, and harmonious dynamic binary translator targeting the Intel® architecture. In *Proceedings of the 8th ACM International Conference on Computing Frontiers*. 1–10.
- [45] Mathias Payer and Thomas R. Gross. 2010. Generating low-overhead dynamic binary translators. In *Proceedings of the 3rd Annual Haifa Experimental Systems Conference*. 1–14.
- [46] GitHub. 2021. PtitSeb: Box64—Linux Userspace x86_64 Emulator with a Twist, Targeted at ARM64 Linux Devices. Retrieved March 30, 2023 from <https://github.com/ptitSeb/box64>
- [47] GitHub. 2003. QEMU, a Generic and Open Source Machine & Userspace Emulator and Virtualizer. Retrieved March 30, 2023 from <https://github.com/qemu/qemu>
- [48] QEMU. 2022. QEMU TCG Plugins—QEMU 7.2.0 Documentation. Retrieved March 30, 2023 from <https://www.qemu.org/docs/master/devel/tcg-plugins.html>
- [49] Rodrigo C. O. Rocha, Dennis Sprokholt, Martin Fink, Redha Gouicem, Tom Spink, Soham Chakraborty, and Pramod Bhatotia. 2022. Lasagne: A static binary translator for weak memory model architectures. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 888–902.
- [50] Ricardo J. Rodríguez, Juan Antonio Artal, and José Merseguer. 2014. Performance evaluation of dynamic binary instrumentation frameworks. *IEEE Latin America Transactions* 12, 8 (2014), 1572–1580.
- [51] Arkaitz Ruiz-Alvarez and Kim Hazelwood. 2008. Evaluating the impact of dynamic binary translation systems on hardware cache performance. In *Proceedings of the 2008 IEEE International Symposium on Workload Characterization*. IEEE, 131–140.
- [52] Filipe Salgado, Tiago Gomes, Adriano Tavares, and Jorge Cabral. 2018. A hardware-assisted translation cache for dynamic binary translation in embedded systems. In *Proceedings of the 2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA '18)*, Vol. 1. IEEE, 307–312.
- [53] Richard L. Sites, Anton Chernoff, Matthew B. Kirk, Maurice P. Marks, and Scott G. Robinson. 1993. Binary translation. *Communications of the ACM* 36, 2 (1993), 69–81.
- [54] Changheng Song, Wenwen Wang, Pen-Chung Yew, Antonia Zhai, and Weihua Zhang. 2019. Unleashing the power of learning: An enhanced learning-based approach for dynamic binary translation. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19)*. 77–90.
- [55] Maxwell Souza, Daniel Nicácio, and Guido Araújo. 2010. ISAMAP: Instruction mapping driven by dynamic binary translation. In *Proceedings of the International Symposium on Computer Architecture*. 117–138.

- [56] Tom Spink, Harry Wagstaff, and Björn Franke. 2016. Hardware-accelerated cross-architecture full-system virtualization. *ACM Transactions on Architecture and Code Optimization* 13, 4 (2016), 1–25.
- [57] Tom Spink, Harry Wagstaff, Björn Franke, and Nigel Topham. 2014. Efficient code generation in a region-based dynamic binary translator. In *Proceedings of the 2014 SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems*. 3–12.
- [58] Swaroop Sridhar, Jonathan S. Shapiro, and Prashanth P. Bungale. 2007. HDTrans: A low-overhead dynamic translator. *ACM SIGARCH Computer Architecture News* 35, 1 (2007), 135–140.
- [59] Harry Wagstaff, Bruno Bodin, Tom Spink, and Björn Franke. 2017. SimBench: A portable benchmarking methodology for full-system simulators. In *Proceedings of the 2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS '17)*. IEEE, 217–226.
- [60] Jun Wang, Jianmin Pang, Liguang Fu, Zheng Shan, Feng Yue, and Jiahao Zhang. 2018. A binary translation backend registers allocation algorithm based on priority. In *Geo-Spatial Knowledge and Intelligence*. Communications in Computer and Information Science, Vol. 849. Springer, 414–425.
- [61] Wenwen Wang, Stephen McCamant, Antonia Zhai, and Pen-Chung Yew. 2018. Enhancing cross-ISA DBT through automatically learned translation rules. *ACM SIGPLAN Notices* 53, 2 (2018), 84–97.
- [62] Wenwen Wang, Jiacheng Wu, Xiaoli Gong, Tao Li, and Pen-Chung Yew. 2018. Improving dynamically-generated code performance on dynamic binary translators. *ACM SIGPLAN Notices* 53, 3 (2018), 17–30.
- [63] Wenwen Wang, P. Yew, Antonia Zhai, and Stephen McCamant. 2016. A general persistent code caching framework for dynamic binary translation (DBT). In *Proceedings of the USENIX Annual Technical Conference*. 591–603.
- [64] Wenwen Wang, Pen-Chung Yew, Antonia Zhai, Stephen McCamant, Youfeng Wu, and Jayaram Bobba. 2017. Enabling cross-ISA offloading for COTS binaries. In *Proceedings of the 15th Annual International Conference on Mobile Systems, Applications, and Services*. 319–331.
- [65] Andrew Waterman, Yunsup Lee, David Patterson, and Krste Asanovic. 2014. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2*. RISC-V Foundation.
- [66] Song Wei, Kun Zhang, and Bibo Tu. 2019. HyperBench: A benchmark suite for virtualization capabilities. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 3, 2 (2019), 1–22.
- [67] Weiwu Hu, Wenxiang Wang, Ruiyang Wu, Huandong Wang, Lu Zeng, Chenghua Xu, Xiang Gao, and Fuxin Zhang. 2023. Loongson instruction set architecture technology. *Journal of Computer Research and Development* 60 (2023), 2–16.
- [68] Pingpeng Yuan, Chong Ding, Long Cheng, Shengli Li, Hai Jin, and Wenzhi Cao. 2010. VITS Test Suit: A micro-benchmark for evaluating performance isolation of virtualization systems. In *Proceedings of the 2010 IEEE 7th International Conference on E-Business Engineering*. 132–139.

Received 19 June 2023; revised 29 October 2023; accepted 4 January 2024